

THORN-2 Technical Reference Manual

aTan

May 30, 2026

Abstract

This document is a technical reference for the *Thorn-2* (Stylised: *P-2*) computer. The *P-2* is an RV64i compatible CPU and supporting system created largely from 74-series logic chips, with the goal of making a fully-pipelined system capable of very-high-speed execution. It also serves as a proof of [in]sanity for the creator.

Contents

1	Introduction	1
1.1	Pipelining	1
1.1.1	Why and How <i>P-2</i> Differs From Classic RISC	1
1.2	Hazard Avoidance	2
1.2.1	Adding Stalls	2
1.2.2	Operand Forwarding	3
2	Program Counter	4
3	Decode Logic	4
4	Immediate Value Generator	4
5	Registers	4
5.1	Theory of Operation	5
5.1.1	Write Phase, ϕ_1	5
5.1.2	Read Phase: ϕ_2	6
5.1.3	Output Buffers	7
5.2	Control Logic	8
6	ALU	8
6.1	Adder	9
6.1.1	Method	9
6.1.2	Modified Carry-Lookahead	10
6.1.3	Doing the Adding	11
6.1.4	Condition Checking	12
6.2	Logicer	13
6.2.1	Logic from MUXs	14
6.2.2	Setting up the MUX	15
6.3	Shifter	16
6.3.1	Bit Order Reverser	16
6.3.2	Shift Stages	16
	References	17
A	Various Logic Expansions and Definitions	18
A.1	Adder Conditions	18

List of Figures

1	Execution flow through a pipelined ALU. I_n refers to line number in code listing.	4
2	The <i>Thorn-2</i> register file.	6

3	Register file timing diagram.	7
4	Layout of the adder circuit. Logic shared between similar signals combined for space.	9
5	Simplified overview of the adder logic.	10
6	Diagram of the Logicer.	13
7	Wiring of a 4:1 mux as used in the logic section.	14
8	Diode ROM storing the truth table function, F_0, F_1, F_2, F_3 ; these are pulled <i>up</i> as the outputs of a 74x139 are active low.	15

List of Tables

1	RISC-V calling convention	5
2	Register Card control signal definitions.	8
3	Contents of the Condition RAM	11
4	Contents of the Carry RAM	12
5	ALU adder EEPROM contents	12
6	Formulae for ALU result conditons	13
7	Truth tables of the three defined logic operations	14
8	Comparison of the number of components used in designs with (a) separate logic paths and (b) MUX-based logic	15

Listings

1	Some example code which denerates lots of stalls.	2
---	---	---

1 Introduction

Thorn-2 is a RISC-V CPU, designed to be fully compatible with the RV64i (integer only) standard module as defined in the RISC-V Instruction Set Architecture (RV-ISA) specification [1]. It is a fully-pipelined design with 5 parallel stages (see §Pipelining) with hazard avoidance (§Hazard Avoidance) and operand forwarding (§Operand Forwarding).

This project has some fairly nebulous goals, but the general idea is to learn how CPUs work internally and make something interesting. They are here itemised:

- Implement a RISC-V CPU
- Utilise as few non-74 series chips as possible (no FPGA)
- Design the circuitry "by hand"
- Achieve ~40 MHz operation
- Have fun :3

Please don't tell me I'm doing this inefficiently or that there are better ways to implement things - I'm well aware.

1.1 Pipelining

1.1.1 Why and How *P-2* Differs From Classic RISC

The pipeline structure of *P-2* differs from the traditional RISC one in that there are several more stages. These can be grouped into largely the same functional blocks, but spread across more stages; a setup reached after much design work on *Thorn-1*.

Classic RISC is split into five stages: Fetch, Decode, Execute, Memory and Writeback; with every instruction passing through each stage in turn (with the correct options set). In implementing this, I decided to use SRAM to store the 32 registers and to avoid using asynchronous SRAM. The latter decision was based on the false assumption that this would make the control circuitry easier to design and less error-prone. I maintained the Synchronous SRAM design for quite a while, getting as far as designing the whole control circuit for the registers and starting work on the ALU, but ran into problems with timings.

The SRAMs I had chosen - IS61NLP6432a [2] - were themselves pipelined at three stages: Assert, wait, Read/Write. Each cycle of the CPU required three register accesses (two reads and a write), which was possible to achieve if each cycle was split into four phases, $\phi_0, \phi_1, \phi_2, \phi_3$. This imposed the requirement that every pipeline stage in the CPU must take

```

1 ;Some silly code :3c
2 add a0, a1, a2 ;Use a0 as tally
3 add a0, a0, a3
4 add a0, a0, a4
5 add a0, a0, a5 ;Add lots of numbers in turn
6 ;...

```

Listing 1: Some example code which denerates lots of stalls.

four cycles, which was insufficient for the ALU to perform a 64-bit addition *and* perform conditional inversion of the second operand for subtraction and check conditions. Thus, I decided to go back to the drawing board and start again.

The current design of *P-2*'s pipeline is based around the ALU, rather than the registers. I needed four stages to add numbers, plus two more to condition the inputs and outputs, giving six. Register accesses can be done in one step by using two copies of each register (a poor-man's dual-port RAM) which are written to simultaneously on the rising edge of the clock; then separately read on the falling edge. Now using fast, *asynchronous* SRAM instead.

1.2 Hazard Avoidance

In a pipelined system, there are frequently situations where one instruction depends on the result of its predecessor. In such a case, this results in a **Data Hazard** - what is contained in register X is out of date, and reading this will produce incorrect results.

Another hazard comes in the case that a conditional branch is dependent on the outcome of a previous operation, resulting in a **Control Hazard**. The CPU must either stall new instructions while the condition is evaluated, or speculatively execute code for either the "branch" or "no branch" condition and throw it out if needed.

1.2.1 Adding Stalls

In either hazard case, the simplest - albeit least efficient - solution is to inject NO-OPs (NOPs) after the Decode stage and only continue when there are no remaining hazards. It is, however, trivial to come up with example programs which result in more nops than executed instructions:

When executing the above program, the CPU is forced to insert NOPs between every instruction to allow the addition result to be fully written back to `a0` before the next instruction is executed. A classic RISC-V architecture with this simple hazard avoidance would *actually* execute the code in Listing 2.

```

7 ;Oops, all stalls TwT
8 add a0, a1, a2 ;Use a0 as tally
9 nop           ;Execute in ALU
10 nop          ;Skip memory access
11 nop          ;Write result to a0
12 add a0, a0, a3 ;Next ADD is allowed to continue
13 nop
14 nop
15 nop
16 add a0, a0, a4
17 nop          ;Each nop represents
18 nop          ; 1 stall cycle
19 nop
20 add a0, a0, a5 ;Add lots of numbers in turn
21 ;...

```

Listing 2: Worst case scenario for consecutive adds.

This effectively quarters the useful throughput of the CPU in the worst case, though most of the time this isn't likely to occur. Using this method, the number of NOPs scales linearly with the number of stages in the pipeline, so there is strong incentive to use a better system. Such a system should be able to feed data from one stage to another in order to skip some of the waiting.

1.2.2 Operand Forwarding

Operand Forwarding is one way to reduce the number of stall cycles introduced with a pipelined system. Here, in the case that an instruction needs data which has not yet been written to a register, but *is available within the pipeline*, that data can be fed forwards early so fewer stalls need to be introduced.

This method becomes very important the longer a pipeline becomes due to a greater number of cycles before the completion of each instruction: there is greater chance that a following instruction is dependent on a previous one *and* that data takes longer to become available.

Assuming the same code snipped as in §Adding Stalls, a perfect system of operand forwarding could reduce the number of NOPs to zero - i.e. what is written is executed exactly.

In *Thorn-2* however, there is an additional complication which arises from the use of a pipelined ALU: the final result is made available in chunks across multiple stages or, should conditions need evaluation, only available after the final stage.

In fig. 1, each of the *Add* stages performs a 16-bit addition of each `int16` chunk of the 64-bit inputs. If operand forwarding were to be used, the output

Invert?	Add 1	Add 2	Add 3	Add 4	Cond.
I2					
I3	I2				
I4	I3	I2			
I5	I4	I3	I2		
	I5	I4	I3	I2	
		I5	I4	I3	I2
			I5	I4	I3
				I5	I4
					I5

Figure 1: Execution flow through a pipelined ALU. I_n refers to line number in code listing.

of *Add 1* would need to be fed back to its inputs for the next instruction. The same would be needed for each subsequent stage too. See §ALU for how this is actually done.

2 Program Counter

The PC’s job is (generally) to increment by 4 every clock cycle to fetch the next instruction in memory. On occasion, it will need to stall in place, load a provided value, or add a constant to its previous value to branch.

While outwardly simple, this is fairly complex to actually implement in 64 bits, especially the branch operation which requires a 12-bit add, then a carry to propagate up the remaining bits. Since there could realistically be any arbitrary sequence of branches, jumps, normal increments or stalls, all of the separate branches need to be done in parallel and selected from at the very end.

3 Decode Logic

4 Immediate Value Generator

5 Registers

RV64i specifies 32 64-bit general-purpose registers and a program counter, PC. x0 is hard-wired to 0x0000 0000 0000 0000; there is no stack pointer. None of the registers are otherwise special, but do have expected uses as per the calling convention [3], shown in table 1.

As each register is effectively the same, they are most easily implemented using SRAM. Learning from *Thorn-1*, I have decided to use the

Table 1: RISC-V calling convention

Register	ABI Mnemonic	Meaning	Preserved across calls?
x0	zero	Zero	— (Immutable)
x1	ra	Return address	No
x2	sp	Stack Pointer	Yes
x3	gp	Global Pointer	— (Unallocatable)
x4	tp	Thread Pointer	— (Unallocatable)
x5-7	t0-2	Temp. Registers	No
x8-9	s0-s1	Callee-saved regs.	Yes
x10-17	a0-7	Argument registers	No
x18-27	s2-11	Callee-saved regs.	Yes
x28-31	t3-6	Temp. Registers	No

IS61WV6416DBLL-10TLI-TR[4]; it's a 64k x 16-bit asynchronous SRAM with a 10 ns access time and single 3.3 V supply.

5.1 Theory of Operation

For every RV64i operation, there are up to two operands, meaning up to two registers may be read each cycle. There may also be a value written into a register. To avoid using multiport RAM, two copies of the registers are needed: **x** and **y**; these two are both written to and read from at the same time. The IS61WV6416 is 16-bit, so there are two sets of four RAM chips total.

Both access to **X** and **Y** (Read/Write) occur within one cycle; writes are completed on the rising edge, ϕ_1 ; and reads on the falling edge, ϕ_2 . This order of operations means the register card doesn't need to latch its input, WB_DATA, or outputs, REG_OUT₁ and REG_OUT₂.

For the purposes of explanation, both RAM banks will be collectively referred to as "R". The logic is otherwise identical between **X** and **Y**, so either could be inserted in R's place.

5.1.1 Write Phase, ϕ_1

Register writes occur on the rising edge of the clock, ϕ_1 , and are controlled only by the external **WRT** signal. Regardless of whether a value is to be written to a register, the address for that 'write' is selected. This setup helps to minimise control circuitry.

Write Enabled, WRT = 1 When **WRT** is high, $\overline{\text{WE}}$ goes low and puts the IS61WV6416 into a write cycle. **R_BUS** is driven to the value on

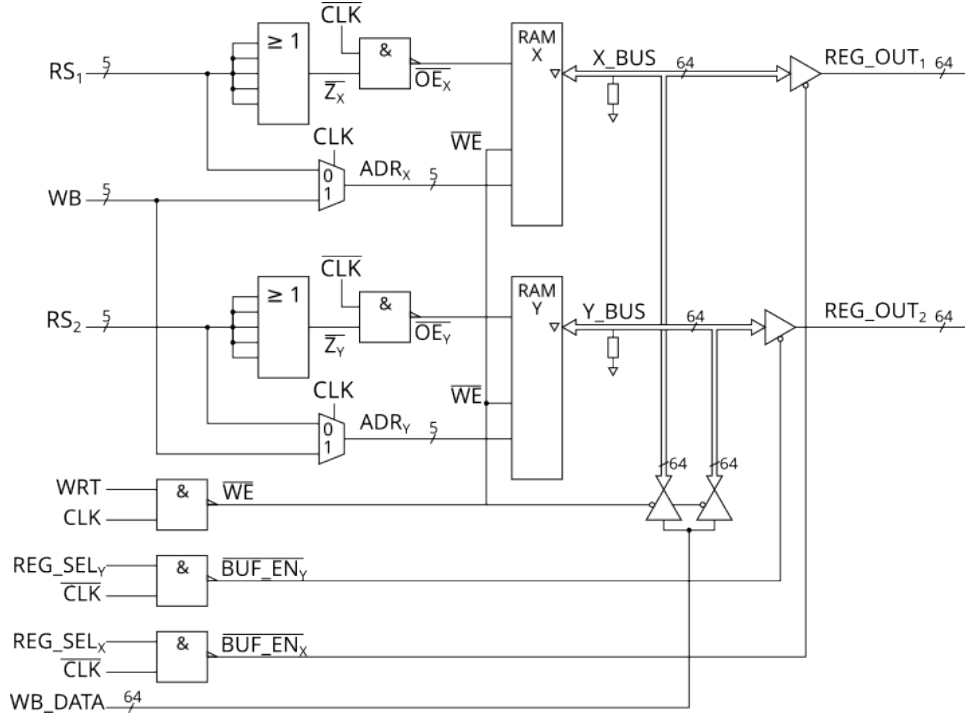


Figure 2: The *Thorn-2* register file.

WB_DATA which is written into the selected register.

Write Disabled, $WRT = 0$ When **WRT** is low, $\overline{WE}$ remains high and the IS61WV6416s have their output disabled via $\overline{OE_R}$. **R_BUS** is then passively pulled down to a value of 0x0000 0000 0000 0000.

5.1.2 Read Phase: ϕ_2

Reads occur on the falling edge of the clock, ϕ_2 , and may give a real register value or a zero depending on the read address (see below). Regardless of whether an IS61WV6416s is actually read from, its address lines, **ADR_R**, are set as if it was.

Not Reading From 'zero', $RS_R \neq 0$ When reading from any register other than $x0$, **Z_R** will be high, setting $\overline{OE_R}$ low during ϕ_2 . **R_BUS** is thus driven to the contents of xR .

Reading From 'zero', $RS_R = 0$ When reading from $x0$, $\overline{Z_R}$ goes low and, during ϕ_2 , $\overline{OE_R}$ stays high, again leaving **R_BUS** to be passively pulled to 0x0000 0000 0000 0000

5.1.3 Output Buffers

Since $P-2$ will be implementing operand forwarding, there are some instances where the input to the ALU isn't necessarily from the register file. Therefore, the register file needs to be "selectable" by the CPU control logic. As such, the outputs of this section, **REG_OUT₁** and **REG_OUT₂** are connected to their respective buses via tri-state buffers.

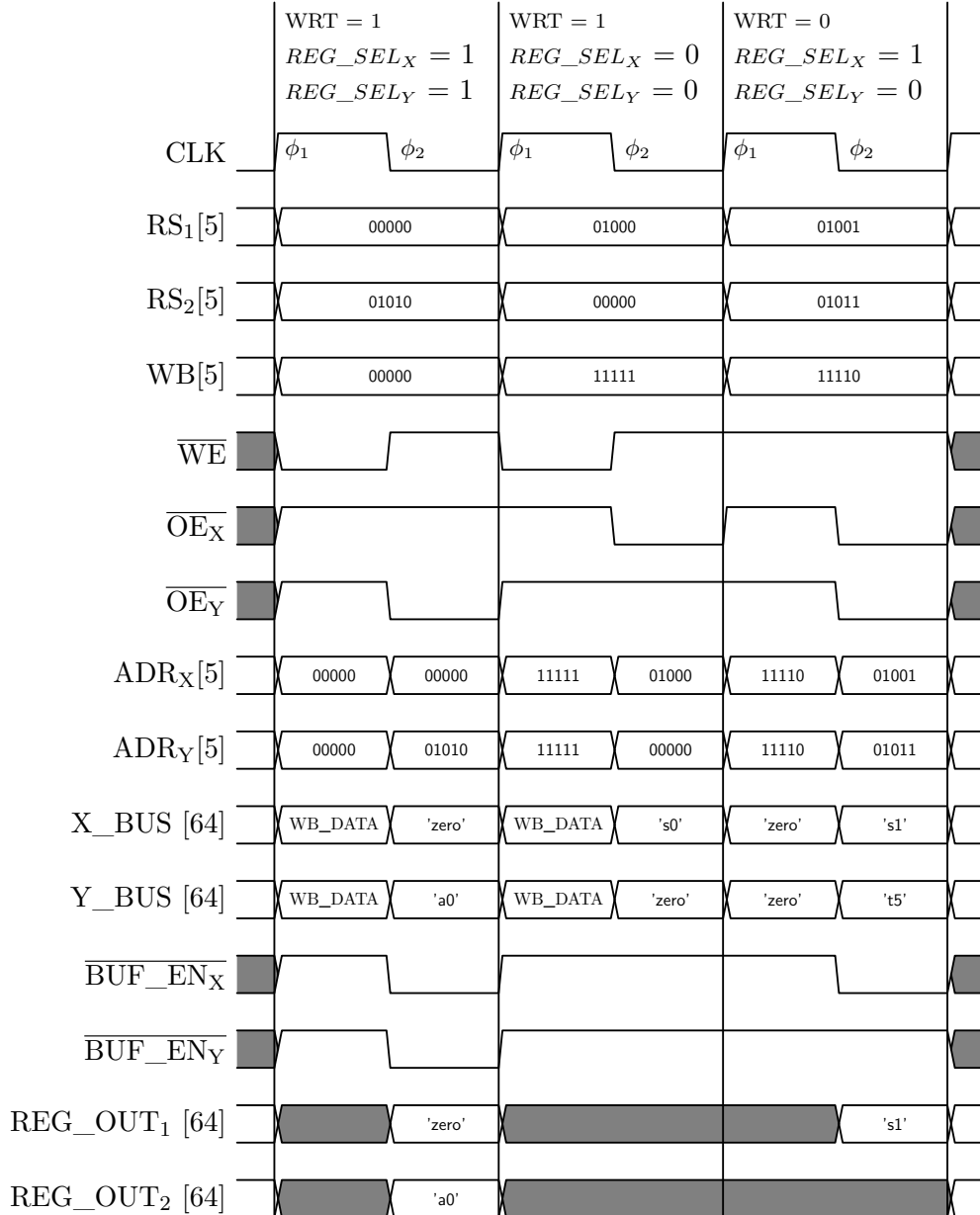


Figure 3: Register file timing diagram.

5.2 Control Logic

Figure 2 shows an overview of the register file. The control signals present are described in table 2.

Table 2: Register Card control signal definitions.

Signal	Equation	Explanation
CLK	—	Clock input
RS₁	—	Read address 1
RS₂	—	Read address 2
WB	—	Write address
WRT	—	Write enable
REG_SEL₁	—	Output X_BUS
REG_SEL₂	—	Output Y_BUS
WB_DATA	—	Data to write this cycle
$\overline{Z_X}$	$\sum_{i=0}^4 (\mathbf{RS}_{1,i})$	Not reading from 'zero'
$\overline{Z_Y}$	$\sum_{i=0}^4 (\mathbf{RS}_{2,i})$	
$\overline{OE_X}$	$\overline{\overline{Z_X} \cdot \overline{CLK}}$	
$\overline{OE_Y}$	$\overline{\overline{Z_Y} \cdot \overline{CLK}}$	
$\overline{WE}$	$\overline{CLK \cdot WRT}$	Write enable signal
REG_OUT₁	$\overline{\overline{CLK} \cdot \overline{REG_SEL_1}}$	Enable output to ALU
REG_OUT₂	$\overline{\overline{CLK} \cdot \overline{REG_SEL_2}}$	

6 ALU

The *Thorn-2*ALU is comprised of three blocks - the Adder, Shifter and Logicer. The Adder computes $A \pm B$; the Shifter shifts A up or down by n bits; and the Logic-er performs the bitwise logic operations $A \cdot B$, $A + B$ and $A \oplus B$. The 64-bit values A and B may be read from registers or immediate values derived from an instruction. Each of the three blocks are contained on separate circuit boards, and run in parallel. Their outputs are calculated for all inputs with the relevant one forwarded to the next pipeline stage.

6.1 Adder

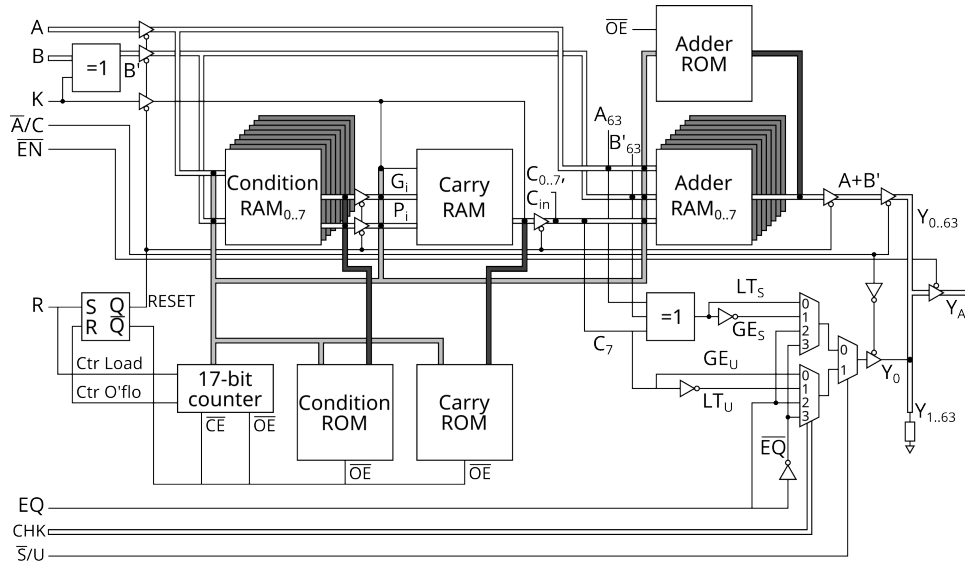


Figure 4: Layout of the adder circuit. Logic shared between similar signals combined for space.

The adder computes $A \pm B$ and optionally checks the result against each of the "flags" for the calculation: $A < B$, $A = B$ and $A \geq B$. In P-1, this procedure was to be pipelined at up to six stages, but resulted in both huge pipelines and complications in feeding partial results back to previous stages, possibly from multiple instructions at multiple different points in their execution. In P-2, however, the entire ALU instead operates in one cycle - the length of this defining the overall cycle length of the CPU.

As you can see from fig. 4, the adder remains one of the most complex parts of the CPU, despite being only a third of the ALU. Using RAM with ~ 8 ns access time, this section has a critical path of ~ 50 ns through the zero check logic. This is markedly better than a more brute-force approach, though still limits the maximum clock frequency of ~ 20 MHz.

6.1.1 Method

Addition and subtraction are both done with the same circuitry, with a pre-conditioning step for the second input, B . Using 2's complement to represent negative numbers, an n -bit number, x , can be negated by inverting its bits and adding 1:

$$-x := (x \oplus [2^n - 1]) + 1 \quad (1)$$

Subtracting **B** from **A** is therefore defined:

$$A - B \equiv A + (-B) \equiv A + (B \oplus [2^n - 1]) + 1 \quad (2)$$

Equation (2) is implemented by bitwise XORing B with the ADD/SUB setting, K , and setting the carry input, C_{in} to the same:

$$K = \begin{cases} 0 & \text{if Adding} \\ 1 & \text{if Subtracting} \end{cases} \quad (3)$$

$$A - B \equiv A + (B \oplus K) + K \quad (4)$$

The XORed B will be called B' from here.

The addition, $A + B' + K$ is a 64-bit operation which would be much too slow if performed as a series of daisy-chained segments due to the access time of the RAM ICs. Even in the best case - where the access time were ≤ 8 ns - a chain of eight in a row would be a total delay of ≤ 64 ns, greatly bottlenecking the speed of the processor. A form of carry-lookahead is therefore used to reduce this delay.

6.1.2 Modified Carry-Lookahead

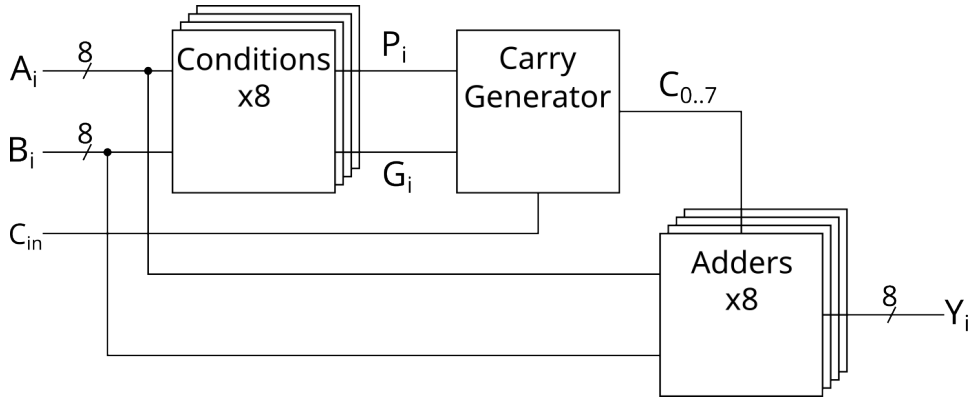


Figure 5: Simplified overview of the adder logic.

A traditional carry-lookahead circuit generates the carry signals from chunks of the inputs, ideally much faster than the same signal would be propagated up a chain of full adders. To do this, additional signals are generated or tapped from the full adder circuits:

$$X_n = A_n \cdot B_n \quad (5)$$

$$S_n = A_n \oplus B_n \quad (6)$$

These additional signals are then used to calculate whether the n -bit chunk generates, G , or propagates, P , a carry.

$$\mathbf{G} = (X_n) + (X_{n-1} \cdot S_n) + \cdots + \left(X_0 \cdot \prod_{i=1}^n (S_i) \right) \quad (7)$$

$$\mathbf{P} = \prod_{i=0}^n (S_i) \quad (8)$$

A single 64Kx8 RAM chip is used as a lookup table for $\mathbf{G}$ and $\mathbf{P}$, both of which are forwarded to another 128Kx8 RAM that generates the carry signals $\mathbf{C}_{0..7}$.

For $n = 0$:

$$C_0 = G_0 + C_{in} \cdot P_0 \quad (9)$$

For $1 \leq n < 8$:

$$C_n = G_n + \sum_{i=1}^n \left(G_{n-i} \cdot \prod_{j=0}^{i-1} (P_{n-j}) \right) + C_{in} \cdot \prod_N (P_n) \quad (10)$$

See §Adder Conditions for expanded definitions.

Using this method, all eight carries can be generated after only two RAM accesses. C_{in} and $C_{0..6}$ are used in the calculation while C_7 is used only for condition checking.

Table 3: Contents of the Condition RAM

$\mathbf{A}_n$	$\mathbf{B}_n$	Address	$\mathbf{G}_n$	$\mathbf{P}_n$
0x00	0x00	0x0000	0	0
0x00	0x01	0x0001	0	0
...				
0x00	0xFF	0x00FF	0	1
0x01	0x00	0x0100	0	0
...				
0x01	0xFE	0x01FE	0	1
0x01	0xFF	0x01FF	1	0
...				
0xFF	0xFF	0xFFFF	1	0

6.1.3 Doing the Adding

The addition itself is "executed" in eight 128Kx8 RAM chips, each programmed at start-up from a single 128Kx8 EEPROM. The RAM is ad-

Table 4: Contents of the Carry RAM

$\mathbf{G}_n$	$\mathbf{P}_n$	$\mathbf{C}_{in}$	Address	$\mathbf{C}$
00000000	00000000	0	0x00000	00000000
00000000	00000001	0	0x00001	00000000
...				
00000000	11111111	0	0x000FF	00000000
00000001	00000000	0	0x00100	00000001
00000001	00000001	0	0x00101	00000001
00000001	00000010	0	0x00102	00000011
...				
11111111	11111111	0	0x0FFFF	11111111
00000000	00000000	1	0x10000	00000000
00000000	00000001	1	0x10001	00000001
...				
11111111	11111111	1	0x1FFFF	11111111

dressed by each byte of the two inputs, $\mathbf{A}_n$ and $\mathbf{B}_n$, and $\mathbf{C}_n$, which points to the result, $\mathbf{Y}_n$.

Table 5: ALU adder EEPROM contents

$\mathbf{A}_n$	$\mathbf{B}_n$	$\mathbf{C}_n$	Address	$\mathbf{Y}_n$
0x00	0x00	0	0x00000	0x00
0x00	0x01	0	0x00001	0x01
...				
0x01	0x00	0	0x00100	0x01
...				
0x00	0x00	1	0x10000	0x01
...				
0xFF	0xFE	1	0x1FFFE	0xFE
0xFF	0xFF	1	0x1FFFF	0xFF

The final result is then either forwarded to the Adder’s output, or changed to 0/1 for the outcome of the SLT/SLTU instructions.

In the case of ADDW/SUBW instructions which sign-extend a 32-bit answer to 64, the upper word is ignored and switched to $\mathbf{Y}$ [31].

6.1.4 Condition Checking

The RISC-V ALU needs to be able to check for three conditions for both signed and unsigned numbers: $\mathbf{A} < \mathbf{B}$, $\mathbf{A} = \mathbf{B}$ and $\mathbf{A} \geq \mathbf{B}$. These are only

valid if the calculation $\mathbf{A} - \mathbf{B}$ was performed. These checks can be performed immediately after the final carry out has been calculated using the equations in table 6.

Table 6: Formulae for ALU result conditons

Condition	Name	Signed	Unsigned
$A < B$	LT	$\overline{A_{63} \oplus B_{63}} \oplus C_7$	$\overline{C_7}$
$A = B$	EQ	$Y = 0$	$Y = 0$
$A \geq B$	GE	$\overline{A < B}_S$	C_7

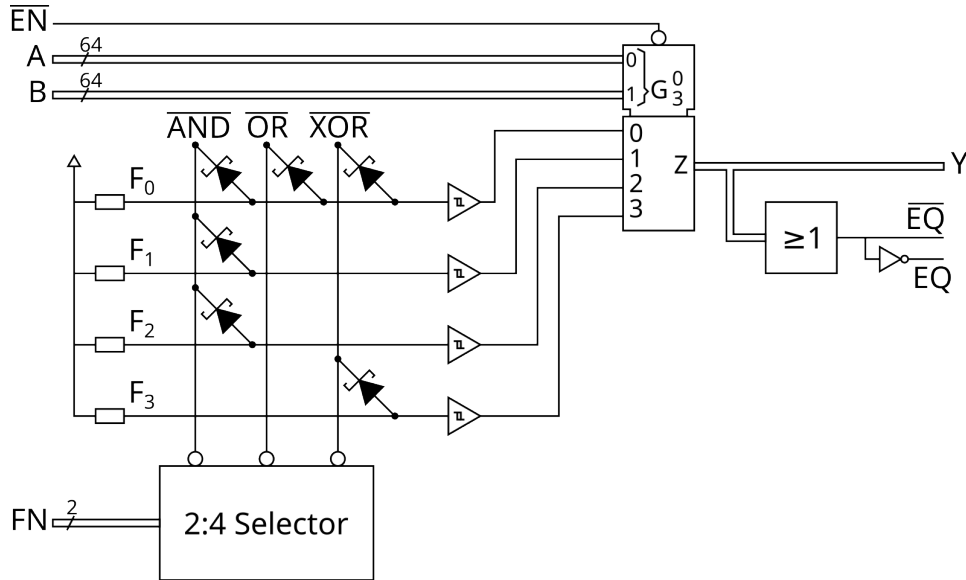
The greater/less than checks are calculated as written, but the equality check is actually calculated using the formula:

$$\overline{EQ} = \sum_{i=0}^{63} (A_i \oplus B_i) \quad (11)$$

In plain english: A and B are equal if $A \oplus B = 0$, which is checked in the Logicer by ORing together all the bits of that result.

The results of the condition check are also forwarded to the program counter to deal with branches.

6.2 Logicer



RISC-V specifies three bitwise logic operations: AND, OR and XOR. The obvious way to do this (74x08 for AND, 74x32 for OR, and 74x86 for XOR), takes 48 chips, plus some extra to choose which answer we want - quite a high number considering what this section does. This can be improved using a slightly more complex setup.

The final output, **Y**, is sent to a tri-state buffer/line driver which is only enabled when the ALU performs a logic operation.

6.2.1 Logic from MUXs

It is possible to construct any 2-input, 1-output logic function from a 4:1 MUX where the four inputs reflect the truth table of the function, and the select lines are the two inputs of the function.

Constructing the logic section this way means the control logic for this section is limited to a small ROM containing the truth tables for the MUXs.

Table 7: Truth tables of the three defined logic operations

A	B	$A \cdot B$	A	B	$A + B$	A	B	$A \oplus B$
0	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	1	1
1	0	0	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0

(a) AND truth table
(b) OR truth table
(c) XOR truth table

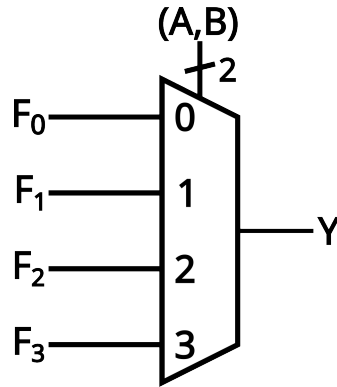


Figure 7: Wiring of a 4:1 mux as used in the logic section.

This method is preferable when considering the number of ICs used:

Table 8: Comparison of the number of components used in designs with (a) separate logic paths and (b) MUX-based logic

Section	#Gates	#ICs	Section	#Parts	#ICs
AND	64	16	ROM Select	1	1
OR	64	16	ROM Diodes	6	—
XOR	64	16	ROM Buffers	12	2
4:1 MUX	64	32	4:1 MUX	64	32
Total	256	80	Total	75	35
(a) Using discrete logic gates			(b) Using MUXs		

6.2.2 Setting up the MUX

The correct truth table is selected using a 2:4 selector which is wired to a small, diode matrix (fig. 8). A 74x1G139 2:4 takes the two-bit function select and the desired bits are set from F_0, F_1, F_2, F_3 , which are then buffered before being sent out to the MUXs. The unused decoder output is left unused and unconnected to the matrix, resulting in a 'function' returning [1,1,1,1].

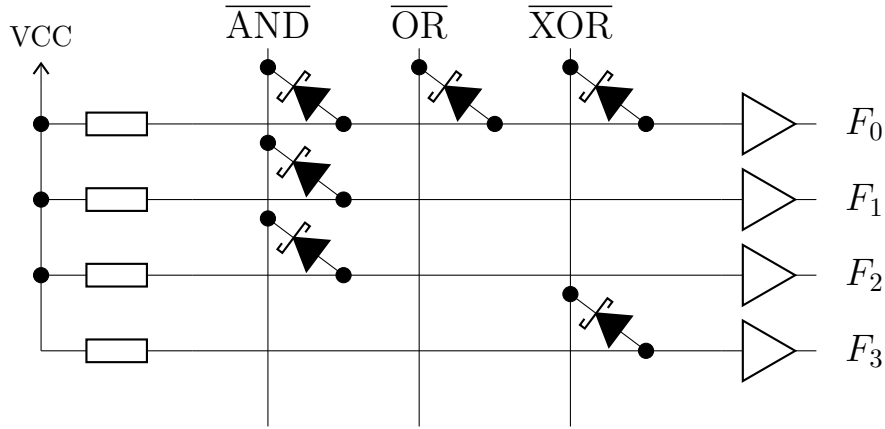


Figure 8: Diode ROM storing the truth table function, F_0, F_1, F_2, F_3 ; these are pulled *up* as the outputs of a 74x139 are active low.

The buffers in fig. 8 serve two purposes: firstly they help with fanout - there are 64 inputs per *Function* line and, for a $4.7k\Omega$ resistor, each can get $\sim 0.7mA$ (this isn't too low for LVC logic, but I would like more headroom); and secondly, they smooth over any issues the diodes may cause.

As there are 64 total MUXs needed for the logic functions, the *Function* lines are buffered four times in parallel so each buffer only needs to drive 16 inputs. The buffers themselves serve two purposes: first to provide enough current to drive all 64 MUXs and second to amplify the F_i signals to counter

the voltage drop from the diodes. With a forward voltage of ~ 0.3 V; this drops the nominal signal level to 3.0 V, only 0.6 V above the minimum V_{ih} of low-voltage CMOS logic chips.

6.3 Shifter

The RISC-V ISA defines several shift operations, each of which can, in one operation, shift the input by an arbitrary number of bits up to X_LEN-1 . In RV64i, additional instructions shift 32-bit values and sign extend up to 64-bits. Both left and right shifts are defined, with left shifts being implemented by reversing the input, shifting, then reversing the result again:

$$A \gg B := \text{REV}(\text{REV}(A) \ll B) \quad (12)$$

This implementation favours left shifts, so any rightward shift will use the method in eq. (12).

Shifts are implemented using a sequential barrel shifter - a series of stages which shift the input a power of two number of places up to 32. Each stage consists of two tri-state buffers, one buffering the previous stage as is, the other buffering the previous output shifted by the next number of bits. The n^{th} stage is then controlled by the n^{th} bit of the shift amount input.

Traditionally, a sequential barrel shifter like this would use multiplexers, but this requires far too many (384 MUXs across 96 ICs) to be practical. The modified version using buffers comes from R. Baruch's RISC-V build [5], expanded to 64-bits.

Using 16-bit buffers in the 74LVC162244, the number of ICs can be reduced to 8 per stage, for a total of 48 - half that of the MUX version.

6.3.1 Bit Order Reverser

In order to shift the incoming number, **A**, rightwards, it must be bit-reversed first. This is accomplished using two tri-state buffers, one buffering the number as is, the other buffering the reversed number.

The reversal is accomplished by "simple" shuffling of the order of traces on the PCB in the following way:

[DIAGRAM OF PCB TRACE REVERSAL]

6.3.2 Shift Stages

References

- [1] A. Waterman and K. Asanovi, Eds., *The risc-v instruction set manual, volume i: User-level isa, document version 20191214-draft*, RISC-V Foundation, Dec. 2019. [Online]. Available: <https://docs.riscv.org/reference/isa/unpriv/unpriv-index.html>.
- [2] I. Integrated Silicon Solution, *Is61nlp6432 ... 64k x 32 ... 2mb, pipeline 'no wait' state bus sram*, Aug. 2005. [Online]. Available: https://www.mouser.co.uk/datasheet/3/3722/1/61NLP_NVP6436A_12818A.pdf.
- [3] K. Cheng and J. Clarke, Eds., *Risc-v abis specification, document version november 26, 2025-draft*, RISC-V International, Nov. 2025. [Online]. Available: <https://riscv-non-isa.github.io/riscv-elf-psabi-doc/>.
- [4] I. Integrated Silicon Solution, *64k x 16 high speed asynchronous cmos static ram*, Mar. 2020. [Online]. Available: https://www.mouser.co.uk/datasheet/3/3722/1/61_64WV6416DAxx_DBxx.pdf.
- [5] R. Baruch, *Lmarv-1 (tangible risc-v) part 4.1: Designing the shifter*, YouTube Video, Jul. 2018. [Online]. Available: <https://www.youtube.com/watch?v=SuUChAF3gaw>.

A Various Logic Expansions and Definitions

A.1 Adder Conditions

Does the n^{th} stage propagate a carry?

$$P_0 = S_{0,0} \cdot S_{0,1} \cdot S_{0,2} \cdot S_{0,3} \cdot S_{0,4} \cdot S_{0,5} \cdot S_{0,6} \cdot S_{0,7} \quad (13)$$

$$P_1 = S_{1,0} \cdot S_{1,1} \cdot S_{1,2} \cdot S_{1,3} \cdot S_{1,4} \cdot S_{1,5} \cdot S_{1,6} \cdot S_{1,7} \quad (14)$$

$$P_2 = S_{2,0} \cdot S_{2,1} \cdot S_{2,2} \cdot S_{2,3} \cdot S_{2,4} \cdot S_{2,5} \cdot S_{2,6} \cdot S_{2,7} \quad (15)$$

$$P_3 = S_{3,0} \cdot S_{3,1} \cdot S_{3,2} \cdot S_{3,3} \cdot S_{3,4} \cdot S_{3,5} \cdot S_{3,6} \cdot S_{3,7} \quad (16)$$

$$P_4 = S_{4,0} \cdot S_{4,1} \cdot S_{4,2} \cdot S_{4,3} \cdot S_{4,4} \cdot S_{4,5} \cdot S_{4,6} \cdot S_{4,7} \quad (17)$$

$$P_5 = S_{5,0} \cdot S_{5,1} \cdot S_{5,2} \cdot S_{5,3} \cdot S_{5,4} \cdot S_{5,5} \cdot S_{5,6} \cdot S_{5,7} \quad (18)$$

$$P_6 = S_{6,0} \cdot S_{6,1} \cdot S_{6,2} \cdot S_{6,3} \cdot S_{6,4} \cdot S_{6,5} \cdot S_{6,6} \cdot S_{6,7} \quad (19)$$

$$P_7 = S_{7,0} \cdot S_{7,1} \cdot S_{7,2} \cdot S_{7,3} \cdot S_{7,4} \cdot S_{7,5} \cdot S_{7,6} \cdot S_{7,7} \quad (20)$$

Does the n^{th} stage generate a carry?

$$G_0 = X_0 + P_0 C_{in} \quad (21)$$

$$G_1 = X_1 + P_1 C_0 + P_1 P_0 C_{in} \quad (22)$$

$$G_2 = X_2 + P_2 C_1 + P_2 P_1 C_0 + P_2 P_1 P_0 C_{in} \quad (23)$$

$$G_3 = X_3 + P_3 C_2 + P_3 P_2 C_1 + P_3 P_2 P_1 C_0 + P_3 P_2 P_1 P_0 C_{in} \quad (24)$$

$$G_4 = X_4 + P_4 (C_3 + P_3 (C_2 + P_2 (C_1 + P_1 (C_0 + P_0 C_{in})))) \quad (25)$$

$$G_5 = X_5 + P_5 (C_4 + P_4 (C_3 + P_3 (C_2 + P_2 (C_1 + P_1 (C_0 + P_0 C_{in})))) \quad (26)$$

$$G_6 = X_6 + P_6 (C_5 + P_5 (C_4 + P_4 (C_3 + P_3 (C_2 + P_2 (C_1 + P_1 (C_0 + P_0 C_{in})))) \quad (27)$$

$$G_7 = X_7 + P_7 (C_6 + P_6 (C_5 + P_5 (C_4 + P_4 (C_3 + P_3 (C_2 + P_2 (C_1 + P_1 (C_0 + P_0 C_{in})))) \quad (28)$$

What is the n^{th} stage's carry in?

$$C_0 = C_{in} \quad (29)$$

$$C_1 = G_0 + P_0 C_{in} \quad (30)$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_{in} \quad (31)$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{in} \quad (32)$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{in} \quad (33)$$

$$C_5 = G_4 + P_4 (G_3 + P_3 (G_2 + P_2 (G_1 + P_1 (G_0 + P_0 C_{in})))) \quad (34)$$

$$C_6 = G_5 + P_5 (G_4 + P_4 (G_3 + P_3 (G_2 + P_2 (G_1 + P_1 (G_0 + P_0 C_{in})))) \quad (35)$$

$$C_7 = G_6 + P_6 (G_5 + P_5 (G_4 + P_4 (G_3 + P_3 (G_2 + P_2 (G_1 + P_1 (G_0 + P_0 C_{in})))) \quad (36)$$

$$C_{out} = G_7 + P_7 (G_6 + P_6 (G_5 + P_5 (G_4 + P_4 (G_3 + P_3 (G_2 + P_2 (G_1 + P_1 (G_0 + P_0 C_{in})))) \quad (37)$$